MIPS Architecture Design

Course Name: EG 121, Computer Architecture - Processor Design

Panav RG
BC2025069
Ananya Singh
BC2025009
Karthik Annadurai
IMT2025016

Introduction

MIPS (Microprocessor without Interlocked Pipelined Stages) is a family of Reduced Instruction Set Computer (RISC) instruction set architectures (ISA) developed by MIPS Computer Systems, now MIPS Technologies. MIPS architecture is widely studied in university and technical school computer architecture courses, and it greatly influenced later RISC architectures such as Alpha. In March 2021, MIPS announced the end of further MIPS architecture development as the company transitions to RISC-V.

Processor

For this assignment, we implemented a MIPS assembly program that performs calculation of fibbonaci using matrix exponentiation, and an optimal way to calculate the power,
and built a fully functional non-pipelined MIPS32 processor simulator in Python that executes the binary machine code generated by MARS. The simulator models all five stages of the classic MIPS datapath IF, ID, EX, MEM, and WB and prints a detailed trace of each stage for every instruction.

Not only this, but it also implements the gcd and sum of array function that we've made later on, which There are snippets of.

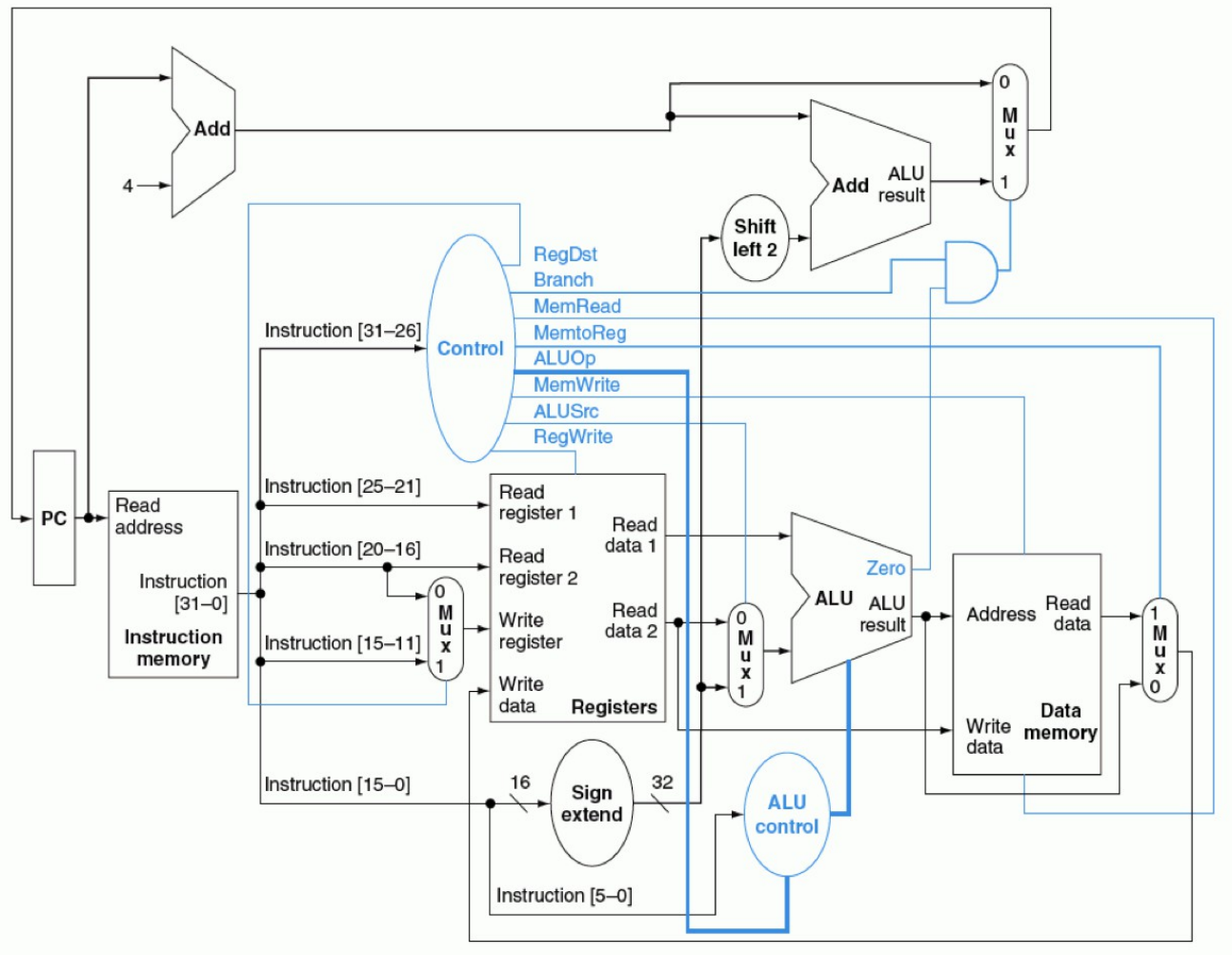

Figure 1: MIPS Architecture

Matrix exponentiation for fibonacci:

This C++ program computes the power of a 2×2 matrix modulo m using fast exponentiation, commonly applied to efficiently calculate Fibonacci numbers.

```
void MM(int a[4], int b[4], int result[4], int m) {
    int p = ((a[0] * b[0]) + (a[1] * b[2])) % m;
    int q = ((a[0] * b[1]) + (a[1] * b[3])) % m;
    int r = ((a[2] * b[0]) + (a[3] * b[2])) % m;
    int s = ((a[2] * b[1]) + (a[3] * b[3])) % m;

    result[0] = p; result[1] = q; result[2] = r; result[3] = s;
}

void copy_matrix(int dest[4], int src[4]) {
    for(int i = 0; i < 4; i++) dest[i] = src[i];
}

int power(int mat[4], int n, int m) {
    n = n - 1;

    int res[4] = {[0]=1, [1]=0, [2]=0, [3]=1};
    int temp[4];

    while(n > 0) {
        if(n % 2 == 1) {
            MM(a: res, b: mat, result: temp, m);
            copy_matrix(dest: res, src: temp);
        }
        MM(a: mat, b: mat, result: temp, m);
        copy_matrix(dest: mat, src: temp);
        n = n / 2;
    }
    return res[0];
}

int main() {
    int n = 19;
    int m = 14;
    int mat[4] = {[0]=1, [1]=1, [2]=1, [3]=0};

    int final_val = power(mat, n, m);
    return final_val;
}
```

Defining each component used:

RegSet:
Creates a register file of the required number of registers and read/write ports to access them.

Register:
It creates a register of the required size and can accept/return values in various formats.

ALU:
It performs all the arithmetic operations and also returns the status of commands like syscalls.

ALU Control:
It maps the Control Unit commands to the required ALU operations.

Control:
It generates control signals for various components like the multiplexers.

Multiplexer:
This creates a multiplexer that controls the data flow based on the current state of its control lines.

Utilities:
It has error handling codes and Enumerations to make our code more readable.

Memory:
It handles everything related to memory and has dedicated separate files for each-Stack, Heap, Text, Data.

Splitter:
It is used during the IF stage to split the bus into smaller segments.

Processor Code :

```python
import sys

def u32(v): return v & 0xFFFFFFFF
def s32(v): v = u32(v); return v - 0x100000000 if v >= 0x80000000 else v
def sext(v, b): return v - (1 << b) if v & (1 << (b-1)) else v

TEXT = 0x00400000
DATA = 0x10010000
SP0  = 0x7FFFFFFC

def fmt(v):
    """Hex for addresses/pointers, decimal for small data values."""
    u = u32(v)
    if u >= 0x00400000:
        return f"0x{u:08X}"
    return str(s32(v))

RNAMES = ["zero","at","v0","v1","a0","a1","a2","a3",
          "t0","t1","t2","t3","t4","t5","t6","t7",
          "s0","s1","s2","s3","s4","s5","s6","s7",
          "t8","t9","k0","k1","gp","sp","fp","ra"]

class RF:
    def __init__(self):
        self.r = [0]*32; self.r[28]=DATA+0x8000; self.r[29]=SP0
        self.HI = self.LO = 0
    def rd(self, i): return self.r[i]
    def wr(self, i, v):
        if i: self.r[i] = u32(v)

class Mem:
    def __init__(self): self.m = {}
    def lw(self, a): return self.m.get(a&~3, 0)
    def sw(self, a, v): self.m[a&~3] = u32(v)
    def lb(self, a):
        w=self.lw(a); sh=(a&3)*8; return (w>>sh)&0xFF
    def sb(self, a, v):
        w=self.lw(a); sh=(a&3)*8
        self.sw(a, (w & ~(0xFF<<sh)) | ((v&0xFF)<<sh))
```

Function Mapping:

```
FUNCT = {
    0x20:"add",0x21:"addu",0x22:"sub",0x23:"subu",
    0x24:"and",0x25:"or",  0x26:"xor",0x27:"nor",
    0x2A:"slt",0x2B:"sltu",
    0x00:"sll",0x02:"srl",0x03:"sra",
    0x04:"sll",0x06:"srl",0x07:"sra",
    0x18:"mult",0x19:"multu",0x1A:"div",0x1B:"divu",
    0x10:"mfhi",0x12:"mflo",0x11:"mthi",0x13:"mtlo",
    0x08:"jr",  0x09:"jalr",
}
ITYPE_OP = {8:"add",9:"addu",0xC:"and",0xD:"or",0xE:"xor",0xA:"slt",0xB:"sltu",0xF:"lui"}
```

Execution Results :

The below screenshots shows the results after running all the immediate files in order.

```
[Cycle 1]  PC=0x00400000
  IF   instr=0x27BDFFF8  >>  addiu $sp, $sp, -8    [addiu   $sp, $sp, -8]
  ID   $rs=$sp=0x7FFFFFFC  $rt=$sp=0x7FFFFFFC  imm=-8
       ctrl: RegWrite=1 ALUSrc=1 MemRead=0 MemWrite=0 Branch=0 Jump=0 RegDst=0 MemToReg=0 ALUOp=3
  EX   addu(0x7FFFFFFC, 0xFFFFFFF8)  =  0x7FFFFFF4   nextPC=0x00400004
  MEM  -
  WB   $sp  <-  0x7FFFFFF4
  -------------------------------------------------------

[Cycle 2]  PC=0x00400004
  IF   instr=0xAFBF0004  >>  sw   $ra, 4($sp)    [sw      $ra,  4($sp)]
  ID   $rs=$sp=0x7FFFFFF4  $rt=$ra=0  imm=4
       ctrl: RegWrite=0 ALUSrc=1 MemRead=0 MemWrite=1 Branch=0 Jump=0 RegDst=0 MemToReg=0 ALUOp=0
  EX   add(0x7FFFFFF4, 4)  =  0x7FFFFFF8   nextPC=0x00400008
  MEM  store  0  ->  MEM[0x7FFFFFF8]
  WB   -
  -------------------------------------------------------

[Cycle 3]  PC=0x00400008
  IF   instr=0xAFB00000  >>  sw   $s0, 0($sp)    [sw      $s0,  0($sp)]
  ID   $rs=$sp=0x7FFFFFF4  $rt=$s0=0  imm=0
       ctrl: RegWrite=0 ALUSrc=1 MemRead=0 MemWrite=1 Branch=0 Jump=0 RegDst=0 MemToReg=0 ALUOp=0
  EX   add(0x7FFFFFF4, 0)  =  0x7FFFFFF4   nextPC=0x0040000C
  MEM  store  0  ->  MEM[0x7FFFFFF4]
  WB   -
  -------------------------------------------------------

[Cycle 4]  PC=0x0040000C
  IF   instr=0x3C011001  >>  lui  $at, 0x1001    [la      $a0, mat]
  ID   $rs=$zero=0  $rt=$at=0  imm=4097
       ctrl: RegWrite=1 ALUSrc=1 MemRead=0 MemWrite=0 Branch=0 Jump=0 RegDst=0 MemToReg=0 ALUOp=3
  EX   lui(0, 4097)  =  0x10010000   nextPC=0x00400010
  MEM  -
  WB   $at  <-  0x10010000
  -------------------------------------------------------

[Cycle 5]  PC=0x00400010
  IF   instr=0x34240000  >>  ori  $a0, $at, 0    [li      $a1, 19]
  ID   $rs=$at=0x10010000  $rt=$a0=0  imm=0
       ctrl: RegWrite=1 ALUSrc=1 MemRead=0 MemWrite=0 Branch=0 Jump=0 RegDst=0 MemToReg=0 ALUOp=3
  EX   or(0x10010000, 0)  =  0x10010000   nextPC=0x00400014
  MEM  -
  WB   $a0  <-  0x10010000
  -------------------------------------------------------
```

MARS Assembly Code:

matrix_power.asm

```
        .data
mat:    .word 1, 1, 1, 0
res:    .word 1, 0, 0, 1
temp:   .word 0, 0, 0, 0
result_msg: .asciiz "Final Result (res[0]): "
newline:    .asciiz "\n"
        .text
        .globl main
main:
        addiu   $sp, $sp, -8
        sw      $ra,  4($sp)
        sw      $s0,  0($sp)
        la      $a0, mat
        li      $a1, 19
        li      $a2, 14
        jal     power
        move    $s0, $v0
        la      $a0, result_msg
        li      $v0, 4
        syscall
        move    $a0, $s0
        li      $v0, 1
        syscall
        la      $a0, newline
        li      $v0, 4
        syscall
        lw      $ra,  4($sp)
        lw      $s0,  0($sp)
        addiu   $sp, $sp,  8
        li      $v0, 10
        syscall
power:
        addiu   $sp, $sp, -28
        sw      $ra,  24($sp)
        sw      $s0,  20($sp)          # s0 = mat address
        sw      $s1,  16($sp)          # s1 = n
        sw      $s2,  12($sp)          # s2 = m
        sw      $s3,   8($sp)          # s3 = &res
        sw      $s4,   4($sp)          # s4 = &temp
        sw      $s5,   0($sp)

        move    $s0, $a0
        move    $s1, $a1
        move    $s2, $a2

        addiu   $s1, $s1, -1
        la      $s3, res
        li      $t0, 1
        sw      $t0,  0($s3)           # res[0] = 1
        sw      $zero, 4($s3)          # res[1] = 0
        sw      $zero, 8($s3)          # res[2] = 0
        sw      $t0, 12($s3)           # res[3] = 1

        la      $s4, temp
power_loop:
        blez    $s1, power_done
        andi    $t0, $s1, 1
        beq     $t0, $zero, power_even
        move    $a0, $s3               # A = res
        move    $a1, $s0               # B = mat
        move    $a2, $s4               # output = temp
        move    $a3, $s2               # m
        jal     MM

        move    $a0, $s3               # dest = res
        move    $a1, $s4               # src  = temp
        jal     copy_matrix            # res = temp

power_even:
        move    $a0, $s0
        move    $a1, $s0
        move    $a2, $s4
        move    $a3, $s2
        jal     MM

        move    $a0, $s0
        move    $a1, $s4
        jal     copy_matrix
```

```
        srl     $s1, $s1, 1             # n = n / 2
        j       power_loop

power_done:
        lw      $v0,  0($s3)            # return res[0]

        lw      $ra,  24($sp)
        lw      $s0,  20($sp)
        lw      $s1,  16($sp)
        lw      $s2,  12($sp)
        lw      $s3,   8($sp)
        lw      $s4,   4($sp)
        lw      $s5,   0($sp)
        addiu   $sp, $sp, 28
        jr      $ra

MM:
        addiu   $sp, $sp, -24
        sw      $ra,  20($sp)
        sw      $s0,  16($sp)           # s0 = &A
        sw      $s1,  12($sp)           # s1 = &B
        sw      $s2,   8($sp)           # s2 = &result
        sw      $s3,   4($sp)           # s3 = m
        sw      $s4,   0($sp)

        move    $s0, $a0
        move    $s1, $a1
        move    $s2, $a2
        move    $s3, $a3

        # result[0] = (A[0]*B[0] + A[1]*B[2]) % m
        lw      $t0,  0($s0)            # A[0]  offset 0
        lw      $t1,  0($s1)            # B[0]  offset 0
        mul     $t0, $t0, $t1
        lw      $t2,  4($s0)            # A[1]  offset 4
        lw      $t3,  8($s1)            # B[2]  offset 8
        mul     $t2, $t2, $t3
        add     $t0, $t0, $t2
        div     $t0, $s3
        mfhi    $t0                     # t0 = sum % m
        sw      $t0,  0($s2)            # result[0]
```

```

        # result[1] = (A[0]*B[1] + A[1]*B[3]) % m
        lw      $t0,  0($s0)            # A[0]  offset 0
        lw      $t1,  4($s1)            # B[1]  offset 4
        mul     $t0, $t0, $t1
        lw      $t2,  4($s0)            # A[1]  offset 4
        lw      $t3, 12($s1)            # B[3]  offset 12
        mul     $t2, $t2, $t3
        add     $t0, $t0, $t2
        div     $t0, $s3
        mfhi    $t0
        sw      $t0,  4($s2)            # result[1]

        # result[2] = (A[2]*B[0] + A[3]*B[2]) % m
        lw      $t0,  8($s0)            # A[2]  offset 8
        lw      $t1,  0($s1)            # B[0]  offset 0
        mul     $t0, $t0, $t1
        lw      $t2, 12($s0)            # A[3]  offset 12
        lw      $t3,  8($s1)            # B[2]  offset 8
        mul     $t2, $t2, $t3
        add     $t0, $t0, $t2
        div     $t0, $s3
        mfhi    $t0
        sw      $t0,  8($s2)            # result[2]

        # result[3] = (A[2]*B[1] + A[3]*B[3]) % m
        lw      $t0,  8($s0)            # A[2]  offset 8
        lw      $t1,  4($s1)            # B[1]  offset 4
        mul     $t0, $t0, $t1
        lw      $t2, 12($s0)            # A[3]  offset 12
        lw      $t3, 12($s1)            # B[3]  offset 12
        mul     $t2, $t2, $t3
        add     $t0, $t0, $t2
        div     $t0, $s3
        mfhi    $t0
        sw      $t0, 12($s2)            # result[3]

        lw      $ra,  20($sp)
        lw      $s0,  16($sp)
        lw      $s1,  12($sp)
        lw      $s2,   8($sp)
        lw      $s3,   4($sp)
        lw      $s4,   0($sp)
        addiu   $sp, $sp, 24
```

```
        jr      $ra
copy_matrix:
        lw      $t0,  0($a1)
        sw      $t0,  0($a0)
        lw      $t0,  4($a1)
        sw      $t0,  4($a0)
        lw      $t0,  8($a1)
        sw      $t0,  8($a0)
        lw      $t0, 12($a1)
        sw      $t0, 12($a0)
        jr      $ra
```

Other implementations:-

GCD function

```cpp
using namespace std;

int sumArray(int* arr, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    return sum;
}

int main() {
    int arr[] = {[0]=3, [1]=7, [2]=2, [3]=9, [4]=4};
    int n = 5;
    int result = sumArray(arr, n);
}
```

GCD function MARS Assembly

```
.data
    result_msg: .asciiz "GCD = "
    newline:    .asciiz "\n"
.text
.globl main
main:
    addiu $sp, $sp, -8
    sw    $ra, 4($sp)
    sw    $s0, 0($sp)

    li    $a0, 48           # a = 48
    li    $a1, 32          # b = 18
    jal   gcd
    move  $s0, $v0

    li    $v0, 4
    la    $a0, result_msg
    syscall

    li    $v0, 1
    move  $a0, $s0
    syscall

    li    $v0, 4
    la    $a0, newline
    syscall

    lw    $ra, 4($sp)
    lw    $s0, 0($sp)
    addiu $sp, $sp, 8

    li    $v0, 10
    syscall
gcd:
gcd_loop:
    beq   $a1, $zero, gcd_done
    div   $a0, $a1
    mfhi  $t0
    move  $a0, $a1
    move  $a1, $t0
    j     gcd_loop
gcd_done:
    move  $v0, $a0
    jr    $ra
```

2)GCD Function

```cpp
#include <iostream>
using namespace std;

int sumArray(int* arr, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    return sum;
}

int main() {
    int arr[] = {[0]=3, [1]=7, [2]=2, [3]=9, [4]=4};
    int n = 5;
    int result = sumArray(arr, n);
    cout << "Sum = " << result << endl;  // expected: 25
    return 0;
}
```

SUM OF ARRAY MARS ASSEMBLY

```
.data
    array:      .word 3, 7, 2, 9, 11
    n:          .word 5
    msg_sum:    .asciiz "Sum = "
    newline:    .asciiz "\n"

.text
.globl main

main:

    addiu   $sp, $sp, -8
    sw      $ra, 4($sp)
    sw      $s0, 0($sp)


    la      $a0, array
    lw      $a1, n

    jal     sumArray
    move    $s0, $v0

    la      $a0, msg_sum
    li      $v0, 4
    syscall

    move    $a0, $s0
    li      $v0, 1
    syscall

    la      $a0, newline
    li      $v0, 4
    syscall

    lw      $ra, 4($sp)
    lw      $s0, 0($sp)
    addiu   $sp, $sp, 8
    li      $v0, 10
    syscall
```

```
sumArray:
    addiu   $sp, $sp, -8
    sw      $ra, 4($sp)
    sw      $s0, 0($sp)

    move    $s0, $a0
    li      $v0, 0
    li      $t0, 0

loop:
    beq     $t0, $a1, done

    sll     $t1, $t0, 2
    add     $t1, $s0, $t1
    lw      $t2, 0($t1)
    add     $v0, $v0, $t2

    addiu   $t0, $t0, 1
    j       loop

done:
    lw      $ra, 4($sp)
    lw      $s0, 0($sp)
    addiu   $sp, $sp, 8
    jr      $ra
```